

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_16

16. Observer Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Observer pattern.

GoF Definition

It defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Concept

In this pattern, there are two types of objects. One is an observer and the other is the subject. What is an observer? In simple words, it is an object that needs to be informed when interesting stuff happens in another object. The object about whom an observer is interested is called the subject.

Normally many observers observe a particular subject. Since the observers are interested in the changes in a subject, they register themselves to get the notifications from the subject. When they lose interest in the subject, they simply unregister from the subject. Sometimes this model is referred to as the Publisher-Subscriber (Pub-Sub) model.

The whole idea can be summarized as follows: using this pattern, an object (subject) can send notifications to multiple observers (a set of objects) at the same time. Observers can decide how to respond/react to these notifications. You can visualize the scenarios with the following diagrams.

In step 1, three observers request notifications from a subject (see Figure [16-1](#)).

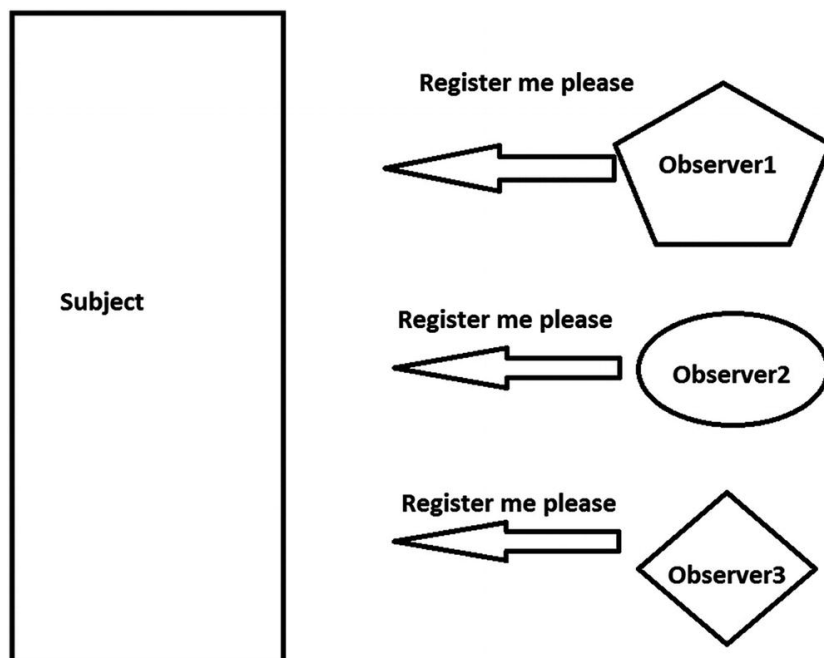


Figure 16-1 Step 1

In step 2, the subject can grant the requests; in other words, a connection is established (Figure [16-2](#)).

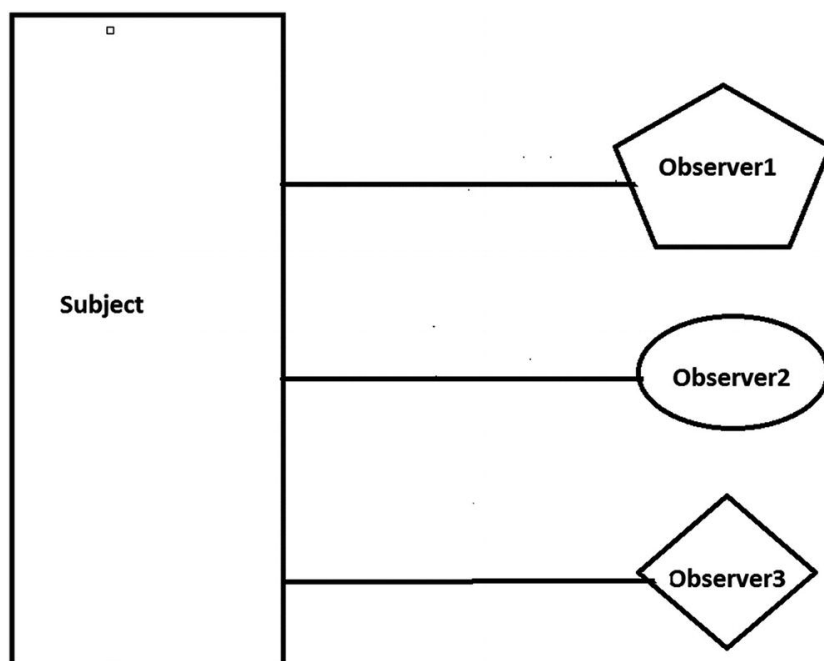


Figure 16-2 Step 2

In step 3, the subject sends notifications to the registered users (see Figure [16-3](#)).

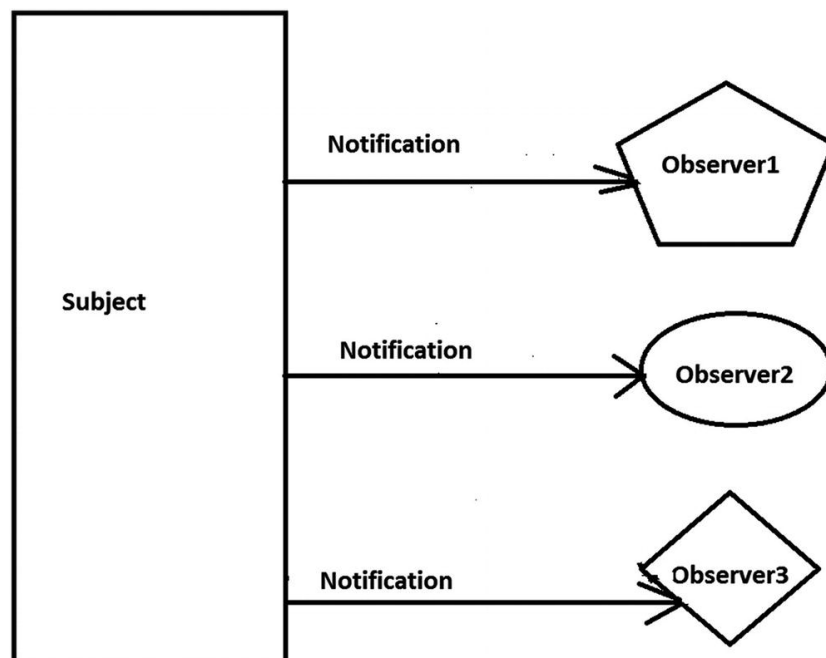


Figure 16-3 Step 3

In step 4 (optional), Observer2 does not want to get further notifications and requests to unregister themselves (or the subject doesn't want to keep Observer2 in their notification list due to some specific reason, so they unregister Observer2). So, the connection between the subject and Observer2 is lost (see Figure [16-4](#)).

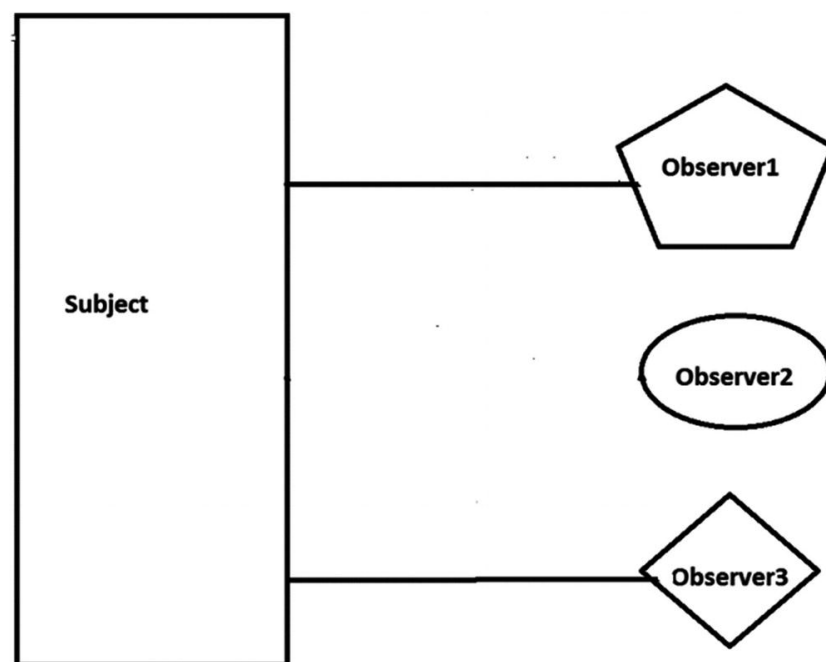


Figure 16-4 Step 4

In step 5, from now on, only Observer1 and Observer3 get notifications from the subject (see Figure [16-5](#)).

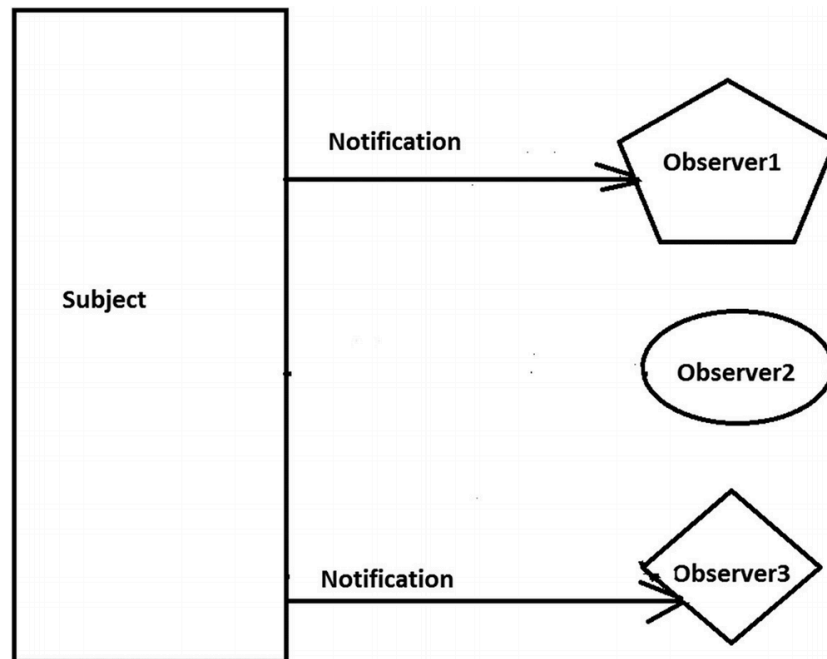


Figure 16-5 Step 5

Real-Life Example

Think about a celebrity who has many followers on social media. Each of these followers wants to get all the latest updates from their favorite celebrity. So, they follow the celebrity until their interest wanes. When they lose interest, they simply unfollow that celebrity. Think of each of these fans or followers as an observer and the celebrity as the subject. A subject can allow any number of observers including 0.

Similarly, you can say that an email subscription or any newsletter subscription follows this pattern.

Computer World Example

In the world of computer science, consider a simple UI-based example where the UI is connected to a database. A user can execute a query through that UI, and after searching the database, the result is returned to the UI. With this pattern, you segregate the UI from the database. If a change occurs in the database, the UI should be notified so that it can update its display according to the change.

To simplify this scenario, assume that you are the person responsible for maintaining a particular database in your organization. Whenever there is a change made to the database, you want to get a notification so that you can take action if necessary.

You can see the presence of this pattern in any event-driven software. Modern languages like Java and C# have built-in support for handling these events following this pattern. These constructs will make your life easier. In this context, note the following points:

- In Java programming, you see the use of event listeners. These listeners are nothing but observers only. Before Java 9, there were ready-

made constructs such as the `Observable` class and the `Observer` interface. `Observable` can have multiple observers who need to implement the `Observer` interface.

- On further investigation, you see that the `Observer` interface has an update method as follows: `void update(Observable o, Object arg)`. This method is invoked whenever a change occurs in the observed object. Your application needs to call the `Observable` object's `notifyObservers()` method to notify the change of the observers. The methods `addObserver(Observer o)` and `deleteObserver(Observer o)` are used to add an observer and delete an observer, similar to our `register` and `unregister` methods.
- This `Observable` class was used to represent the observable objects or “data” in the model-view paradigm. Java developers used to subclass it to represent an object that the application wants to have observed.

Note Java 9 onwards they are deprecated. The Eclipse IDE shows the following description: This class (`Observable`) and the `Observer` interface have been deprecated. The event model supported by `Observer` and `Observable` is quite limited, the order of notifications delivered by `Observable` is unspecified, and state changes are not in one-for-one correspondence with notifications. For a richer event model, consider using `java.beans` package. For reliable and ordered messaging among threads, consider using one of the concurrent data structures in the `java.util.concurrent` package. For reactive streams style programming, see the `java.util.concurrent.Flow` API.

- Following the previous suggestion, let's investigate the alternatives. In Java 9 onwards, the `Flow` class is introduced. Here you see the presence of static interfaces such as `Publisher<T>`, `Subscriber<T>`, and `Subscription`. These constructs can be used as an alternative to the `Observer` design pattern.
- If you are already familiar with the .NET Framework, you know that in C#, you already have generic `System.IObservable<T>` and `System.IObserver<T>` interfaces where the generic type parameter is used to provide notifications.

Implementation

In this example, you see two inheritance chains, one for the subject and one for the observers. The observer hierarchy starts with the `Observer` interface. Two concrete types of `Employee` and `Customer` implement this interface. *I created two different types of observers to demonstrate that they need not belong to the same class. Consider a real-world example. A company has both employees and customers who can purchase the company stocks. As a result, they like to get alerts when the stock price changes. In this case, both customers and employees are the observers and the company is the subject.*

Inside the client code, you'll see four observer instances (`roy`, `kevin`, `bose`, and `jacklin`). There are two subject instances too; one is for `ABC Ltd.` and the other is for the `XYZ Co.` Let us call them as `abcLtd` and `xyz-Co`, respectively.

Each subject maintains a list of all of its registered users. The observers receive a notification when the stock price changes in the subject(s).

Initially, three observers (`Roy`, `Kevin`, and `Bose`) register themselves to get notifications from `ABC Ltd.` (when the stock price is \$5). So, in the initial phase, all of them received notifications. Later `Kevin` loses interest. When the subject becomes aware of this, it removes `Kevin` from its observer list. At this time, only `Roy` and `Bose` receive notifications (when the stock price is \$50). But `Kevin` changes his mind later and wants to get notifications again, so the company `ABC Ltd.` registers him again. This is why, when the stock price jumps to \$100, all three observers receive notifications.

Later you see another company named `XYZ Co.` `Roy` and `Jacklin` are registered in its observer list. So, when this company sets its stock price to \$500, both `Roy` and `Jacklin` receive the notification.

Now, look into the code. Here is the `Observer` interface, which has the following definition:

```
interface Observer {
    void getNotification(Company company);
    String getObserverName();
}
```

Two concrete classes named `Employee` and `Customer` are used to show that you can have different types of observers. These classes implement the `Observer` interface. They are similar. Here is one of them:

```
//Observer type-1: These are employees
class Employee implements Observer {
    String nameOfObserver;
    public Employee(String name) {
        this.nameOfObserver = name;
    }

    @Override
    public void getNotification(Company company) {
        System.out.print(nameOfObserver + " has received an
                           alert from " +company.getName());
        System.out.println("The current stock price is:$" +
                           company.getStockPrice());
    }
    @Override
    public String getObserverName() {
        return nameOfObserver;
    }
}
```

```
}
```

To represent subjects, let's start with an abstract class. Notice that this class contains three methods called `register(...)`, `unRegister(...)`, and `notifyRegisteredUsers()`. As per the names, these methods are used to register an observer, unregister an observer, and notify all registered observers, respectively. In addition, this class maintains a list of observers. The following code segment with supporting comments help you understand this class:

```
abstract class Company {
    List<Observer> observerList = new ArrayList<Observer>();
    // Name of the subject
    private String name;
    public Company(String name) {
        this.name = name;
    }
    public String getName() {
        return this.name;
    }
    // For the stock price
    private int stockPrice;
    public int getStockPrice() {

        return this.stockPrice;
    }
    public void setStockPrice(int stockPrice) {
        this.stockPrice = stockPrice;
        // The stock price is changed.
        // So notify observer(s).
        notifyRegisteredUsers();
    }
    // To register an observer
    abstract void register(Observer o);
    // To Unregister an observer
    abstract void unRegister(Observer o);
    // To notify registered users
    abstract void notifyRegisteredUsers();
}
```

Concrete class `SpecificCompany` extends the `Company` class and provides the specific implementations as follows:

```
class SpecificCompany extends Company {
    public SpecificCompany (String name) {
        super(name);
    }
    @Override
    void register(Observer anObserver) {
        observerList.add(anObserver);
        System.out.println(this.getName()+ " registers "+
                           anObserver.getObserverName());
    }
    @Override
    void unRegister(Observer anObserver) {
```

```

        observerList.remove(anObserver);
        System.out.println(this.getName()+ " unregisters "+
                           anObserver.getObserverName());
    }
    // Notify all registered observers.
    @Override
    public void notifyRegisteredUsers() {
        for(Observer observer: observerList) {
            observer.getNotification(this);
        }
    }
}

```

There's a constructor inside the subject. The constructor is as follows:

```

public Company(String name) {
    this.name = name;
}

```

This constructor represents different subjects. So, inside client code, you will see the following lines with comments:

```

System.out.println("Working with the company: Abc Ltd.\n");
Company abcLtd = new SpecificCompany("ABC Ltd.");
// Remaining code skipped

```

The remaining code is easy to understand. You can follow the supportive comments if required.

NoteIn this example, the subject is responsible for registering and/or un-registering an observer. It is because I closely follow the built-in Java constructs where the subject (`Observable` class) does the same, but the `Observer` interface has only the update method. It means that the `Observable` class is only responsible for the observer's registration or unregistration. But you can make an application that allows an observer to register themselves to get a notification from a subject, too. You will see a discussion on this topic later.

Class Diagram

Figure [16-6](#) shows the class diagram.

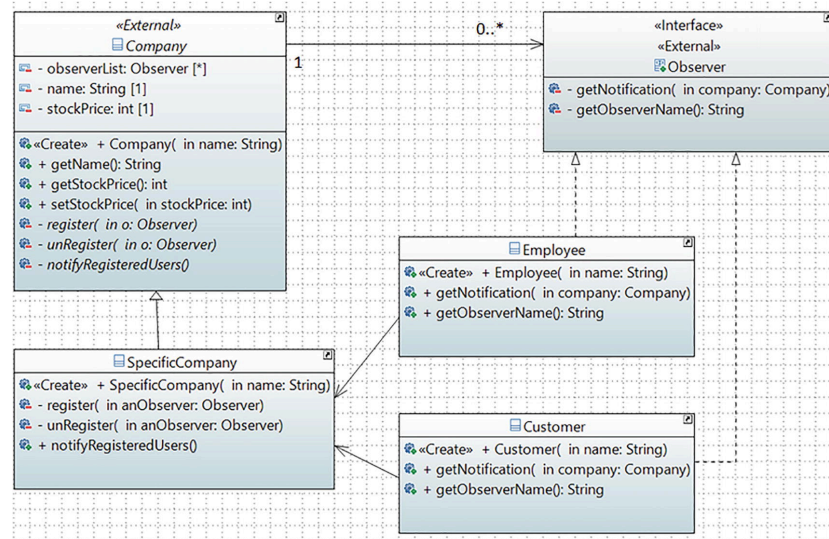


Figure 16-6 The class diagram

Package Explorer View

Figure 16-7 shows the high-level structure of the program. `Employee.java` is similar to `Customer.java`. To make the diagram short, I did not expand it again in the Package Explorer.



Figure 16-7 Package Explorer view

Demonstration

All the classes and the `Observer` interface are stored inside the package `jdp3e.observer.implementation_1`. Here's the complete demonstration:

```
// Observer.java
interface Observer {
    void getNotification(Company company);
    String getObserverName();
}

// Employee.java
// Observer type-1: These are employees
class Employee implements Observer {
    String nameOfObserver;
    public Employee(String name) {
        this.nameOfObserver = name;
    }
    @Override
    public void getNotification(Company company) {
        System.out.print(nameOfObserver + " has received an
                           alert from " +company.getName());
        System.out.println("The current stock price is:$" +
                           company.getStockPrice());
    }
    @Override
    public String getObserverName() {
        return nameOfObserver;
    }
}

// Customer.java
// Observer type-2: These are customers
class Customer implements Observer {
    String nameOfObserver;
    public Customer(String name) {
        this.nameOfObserver = name;
    }
    @Override
    public void getNotification(Company company) {

        System.out.print(nameOfObserver + " is notified
                           from " +company.getName());
        System.out.println("Its current stock price is:$"
                           + company.getStockPrice());
    }
    @Override
    public String getObserverName() {
        return nameOfObserver;
    }
}

// Company.java
import java.util.ArrayList;
import java.util.List;
abstract class Company {
    List<Observer> observerList = new ArrayList<Observer>();
    // Name of the subject
    private String name;
    public Company(String name) {
```

```

        this.name = name;
    }
    public String getName() {
        return this.name;
    }
    // For the stock price
    private int stockPrice;
    public int getStockPrice() {
        return this.stockPrice;
    }
    public void setStockPrice(int stockPrice) {
        this.stockPrice = stockPrice;
        // The stock price is changed.
        // So, notify observer(s).
        notifyRegisteredUsers();
    }
    // To register an observer
    abstract void register(Observer o);
    // To Unregister an observer
    abstract void unRegister(Observer o);
    // To notify registered users
    abstract void notifyRegisteredUsers();
}
// SpecificCompany.java
class SpecificCompany extends Company {
    public SpecificCompany (String name) {
        super(name);
    }
    @Override
    void register(Observer anObserver) {
        observerList.add(anObserver);
        System.out.println(this.getName()+ " registers " + anObserver.getObserverName());
    }
    @Override
    void unRegister(Observer anObserver) {
        observerList.remove(anObserver);
        System.out.println(this.getName()+ " unregisters " + anObserver.getObserverName());
    }
    // Notify all registered observers.
    @Override
    public void notifyRegisteredUsers() {
        for(Observer observer: observerList) {
            observer.getNotification(this);
        }
    }
}
// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Observer Pattern Demonstration.***\n");
        // We have 4 different observers.
        Observer roy = new Employee("Roy");
        Observer kevin = new Employee("Kevin");
        Observer bose = new Customer("Bose");
        Observer jacklin = new Customer("Jacklin");
        System.out.println("Working with the company: Abc Ltd.\n");
    }
}

```

```

        Company abcLtd = new SpecificCompany("ABC Ltd. ");
        // Registering the observers - Roy, Kevin, Bose
        abcLtd.register(roy);
        abcLtd.register(kevin);
        abcLtd.register(bose);
        System.out.println(" ABC Ltd.'s current stock price is $5.");
        abcLtd.setStockPrice(5);
        System.out.println("-----");
        // Kevin doesn't want to get further notification.
        System.out.println("\nABC Ltd. is removing Kevin
                           from the observer list now.");
        abcLtd.unregister(kevin);
        // No notification is sent to Kevin any more.
        System.out.println("\n ABC Ltd.'s new stock price

is $50.");
        abcLtd.setStockPrice(50);
        System.out.println("-----");
        System.out.println("\nKevin registers again to get
                           notification from ABC Ltd.");
        abcLtd.register(kevin);
        System.out.println("\n ABC Ltd.'s new stock price is $100.");
        abcLtd.setStockPrice(100);
        System.out.println("-----");
        System.out.println("\n Working with another company: XYZ Co.");
        // Creating another company
        Company xyzCo = new SpecificCompany("XYZ Co. ");
        // Registering the observers-Roy and Jacklin
        xyzCo.register(roy);
        xyzCo.register(jacklin);
        System.out.println("\nXYZ Co.'s new stock price is $500.");
        xyzCo.setStockPrice(500);
    }
}

```

Output

Here's the output:

```

***Observer Pattern Demonstration.***
Working with the company: Abc Ltd.
ABC Ltd. registers Roy
ABC Ltd. registers Kevin
ABC Ltd. registers Bose
  ABC Ltd.'s current stock price is $5.
Roy has received an alert from ABC Ltd. The current stock price is:$5
Kevin has received an alert from ABC Ltd. The current stock price is:$5
Bose is notified from ABC Ltd.Its current stock price is:$5
-----
ABC Ltd. is removing Kevin from the observer list now.
ABC Ltd. unregisters Kevin
  ABC Ltd.'s new stock price is $50.
Roy has received an alert from ABC Ltd. The current stock price is:$50
Bose is notified from ABC Ltd.Its current stock price is:$50
-----
Kevin registers again to get notifications from ABC Ltd.

```

```
ABC Ltd. registers Kevin
ABC Ltd.'s new stock price is $100.
Roy has received an alert from ABC Ltd. The current stock price is:$100
Bose is notified from ABC Ltd.Its current stock price is:$100
Kevin has received an alert from ABC Ltd. The current stock price is:$100
-----
Working with another company: XYZ Co.
XYZ Co. registers Roy
XYZ Co. registers Jacklin
XYZ Co.'s new stock price is $500.
Roy has received an alert from XYZ Co. The current stock price is:$500
Jacklin is notified from XYZ Co. Its current stock price is:$500
```

Q&A Session

16.1 If I have only one observer, then I may not need to set up the interface. Is this correct?

Yes. But if you want to follow the pure object-oriented programming guidelines, you should prefer interfaces (or abstract classes) instead of using a concrete class. Aside from this point, usually you have multiple observers and you implement them following the contracts. That's where you get the benefit from this kind of design.

16.2 Can you have different observers that may vary?

Yes. This is why, in the demonstration, I showed different types of observers, such as employees and customers. These observers also work with different subjects (i.e., companies).

16.3 Can you add or remove observers at runtime?

Yes. Notice that at the beginning of the program, to get notifications, Kevin registers himself. In between, he unregisters himself and then reregisters again.

16.4 It appears to me that there are similarities between the Observer pattern and the Chain of Responsibility pattern (Chapter [17](#)). Is this understanding correct?

In an Observer pattern, all registered users get notifications in parallel, but in a Chain of Responsibility pattern, objects in the chain are sequentially notified one by one, and this process continues until an object handles the notification fully (or reaches at the end of the chain). I show the comparisons with diagrams in the “Q&A Session” section of the Chain of Responsibility pattern (see Q&A [17.7](#) in Chapter [17](#)).

16.5 Does this model support one-to-many relationships?

Yes, the GoF definition confirms this. Since a subject can send notifications to multiple observers, this kind of dependency depicts a one-to-many relationship.

16.6 I read previous editions of this book. I understand that before Java 9, you could use the ready-made constructs (for example, `Observable` class and `Observer` interface, etc.). Instead of using them, why did you use custom code to model an Observer Design pattern?

You cannot change ready-made functionalities, but I believe that when you try to implement the concept yourself, you gain an understanding that can help you to better use those ready-made constructs. Let's consider some typical scenarios:

- In Java, `Observable` is a concrete class and it does not implement an interface. We know that programming to an interface is always better than programming to a concrete class when you try implementing polymorphic code.
- You also remember that Java does not allow multiple inheritance through classes. So, when you extend the `Observable` class, you have to keep in mind the restriction. So, it may limit the reuse potential.
- The signature of the `setChanged` method in `Observable` is as follows:

```
protected void setChanged().
```

This means to use it, you needed to subclass the `Observable` class. This violates one of the key design principles that says to *prefer composition over inheritance*.

- Finally, I told you that the `Observable` class and the `Observer` interface have been deprecated since Java 9.

Similarly, if you are familiar with the C# programming language, you may see the use of `System.IObservable<T>` and `System.IObserver<T>` interfaces. What does this mean? It tells us that you need to be familiar with generic programming. Not only that, if you give a closer look at these interfaces, you will see the following:

```
public interface IObservable<out T>
public interface IOObserver<in T>
```

This simply means that you need to be familiar with covariance and contravariance in C#, too. In the beginning, these concepts may seem to be difficult to understand. In *Getting Started with Advanced C#*, I discuss these concepts in detail with code examples.

16.7 What are the key benefits of the Observer pattern?

Here are some key advantages:

- Subjects (companies, in this example) and their registered users (observers) make up a loosely coupled system. They do not need to know each other explicitly. Typically, a subject does not care how an observer reacts to the notification event.
- You do not need to make changes in the subject when you add or remove an observer from its notification lists.
- You can add or remove observers at runtime independently.

16.8 What are the key challenges associated with an Observer pattern?

Here are some key challenges when you implement (or use) this pattern:

- Undoubtedly, memory leak is the greatest concern when you handle events in Java or a similar language such as C#. It is also referred to as the *lapsed listener problem*. In simple words, consider a case when an observer registers with a subject to get event notifications but forgets to unregister the same. It causes some live references to stay in the computer memory. The garbage collector does not collect them and as a result, you see the impact of memory leaks.
- The order of notification is not dependable.
- As the number of observers increases, each update operation becomes costly.

16.9 I have a concern: I see that in your demonstration, the subject is registering and unregistering the observers. But in a real-world application, I want to give these observers more freedom. They should be able to register and unregister themselves, too. How can I make such an application?

It is easy. Let's add two additional methods in the `Observer` interface as follows (notice the changes in bold):

```
interface Observer {
    void getNotification(Company company);
    void registerTo(Company company);
    void unregisterFrom(Company company);
    String getObserverName();
}
```

Here is a sample implementation of a concrete observer:

```
// Observer type-1: These are employees
class Employee implements Observer {
    String nameOfObserver;
    public Employee(String name) {
        this.nameOfObserver = name;
    }
    @Override
    public void getNotification(Company company) {
        System.out.print(nameOfObserver + " has received an alert from " + company.getName() + ": ");
        System.out.println("The current stock price is:$" + company.getStockPrice());
    }
    @Override
    public void registerTo(Company company) {
        company.register(this);
        System.out.println(this.nameOfObserver + " registered himself/herself to " +
            company.getName());
    }
}
```



```

@Override
public void unregisterFrom(Company company) {
    company.unregister(this);
    System.out.println(this.nameOfObserver+ "
                        unregistered himself/herself
                        from "+ company.getName());
}
@Override
public String getObserverName() {
    return nameOfObserver;
}
}

```

No changes are made in the subject hierarchy. Inside the client code, now an observer can register or unregister themselves. Notice the changes in bold in the following code:

```

// Observers
Observer roy = new Employee("Roy");
Observer kevin = new Employee("Kevin");
//other codes, if any
// Observers register themselves
roy.registerTo(xyzCo);
kevin.registerTo(xyzCo);
//some other code
// An observer unregisters himself
roy.unregisterFrom(xyzCo);
//some other code

```

Note I have not put in the validations whether an observer is already registered (or unregistered) to a subject. I leave this kind of exercise for you because our focus is on the design patterns only. I kept an additional implementation on the Apress site to demonstrate how an observer can register to a subject or unregister from the subject. If you want to see this implementation, you can refer to the package `jdnp3e.observer.implementation_2`.

Previous chapter

< [15. Template Method Pattern](#)

Next chapter

[17. Chain of Responsibility Pattern](#) >